The Journal of Open Source Software

PyDPEET: A Python Package for Automated Processing of Battery Measurement Data

Martin Otto [1*¶], **Anton Schlösser** [1*¶], **Daniel Schröder**[1], **Jan Kalisch**[1], **Alexander Hinrichsen**[1], **Cataldo De Simone**[1], **and Julia Kowal** [1¶]

1 TU Berlin, Institute of Energy and Automation, Electrical Energy Storage Technology (EET), Einsteinufer 11, D-10587 Berlin, Germany ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 31 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Experimental battery research produces large amounts of heterogeneous data from laboratory and field applications, often stored in incompatible formats and representations that hinder reproducible analysis. PyDPEET is a Python package for importing, harmonising, structuring, and analysing experimental battery data. It provides a common data representation, automatically identifies operating steps and higher-level experimental sequences, and supports the organisation of individual tests into test series and multi-cell measurement campaigns. Its modular and extensible architecture facilitates the addition of new data readers and analysis functions which enables reusable workflows across measurement systems and reduces the need for device- and project-specific code.

Statement of Need

Experimental battery research generates large amounts of measurement data from a wide range of battery cyclers and electrochemical measurement systems. These systems commonly use specific file formats, column names, units, metadata structures, and representations of experimental procedures. In particular, step and schedule information may differ between measurement systems or may be unavailable altogether, especially for field data. In many research workflows, the analysis is implemented through experiment- or device-specific scripts, making them difficult to transfer between experiments, datasets, and measurement systems (Redondo-Iglesias et al., 2023; Wind et al., 2024).

At the same time, battery characterisation increasingly relies on combinations of different experiments and analysis methods. Subsequent analyses require a consistent representation of base quantities such as voltage, current, and time, as well as of the underlying experimental structure. Reimplementing these for individual projects increases development effort and can reduce reproducibility and comparability between studies (Herring et al., 2020; Redondo-Iglesias et al., 2023).

This implies a need for a reusable and measurement-system-independent processing framework that transforms heterogeneous raw battery measurement data into a consistent and structured representation for further analysis.

State of the Field

Open-source software for battery data processing and analysis spans a wide range of applications and levels of specialisation. Several tools focus on individual characterisation or diagnostic

DRAFT

methods. For example, dedicated tools exist for electrochemical impedance spectroscopy and distribution of relaxation times analysis (Huang & Zeier, 2026; Murbach et al., 2020; Wan et al., 2015), degradation mode analysis (Dubarry et al., 2012; Rehm et al., 2026), and the extraction of model-relevant parameters from techniques such as incremental capacity analysis or galvanostatic intermittent titration (Randall, 2025).

More general frameworks have been developed to process battery cycling data across different experiments and measurement systems, but they differ in the abstractions used to represent experimental procedures and in the extent of the processing workflow they cover. Cellpy (Wind et al., 2024) supports multiple battery cyclers, harmonises their data into a common representation, automatically derives step- and cycle-level information, and provides analysis methods. PyProBE (Holland et al., 2025) similarly converts data from several commonly used cyclers into a standardised representation and organises measurements into cells, procedures, experiments, cycles, and steps. While cycle and event information are represented based on the imported step information, the corresponding experimental information must be manually defined by the user for each test, typically through an accompanying description of the experimental procedure, and is not inferred automatically from the measurement data. BEEP (Herring et al., 2020) follows a different emphasis, combining the structuring of cycling data with feature extraction and workflows for battery lifetime prediction. DATTES (Redondo-Iglesias et al., 2023), implemented in MATLAB and compatible with GNU Octave, converts proprietary cycler data, segments measurements into operating phases, and provides analyses including capacity, resistance, impedance, OCV, ICA, and DVA. The battery-data-toolkit(Ward, 2020) focuses primarily on consistently structured battery datasets, metadata, and reusable post-processing functions, providing a common data representation for subsequent analysis. At a more fundamental level, the Battery Data Format(Battery Data Alliance, 2025) defines a standardised and semantically described representation of battery time-series data and provides tools for importing, converting, validating, and cleaning data from different sources.

Table 1: Comparison of open-source frameworks for battery measurement data processing and analysis. "X" indicates full support, "(X)" partial support, and "–" functionality not provided.

Software	Multi-cycler import	Unified format	Automatic step detection	Automatic sequence detection	General analysis	Diagnostic analysis
Cellpy	X	X	(X)	–	X	X
PyProBE	X	X	–	–	X	X
BEEP	X	X	–	–	X	(X)
DATTES	X	X	(X)	(X)	X	X
battery-data-toolkit	(X)	X	–	–	X	–
Battery Data Format	X	X	–	–	(X)	–
PyDPEET	X	X	X	X	X	X

The direct comparison from Table 1 distinguishes the frameworks according to their support for data import and harmonisation, representation of experimental procedures, automatic reconstruction of operating steps, and extend of analysis functionality. Most frameworks support the first aspect, whereas the representation and reconstruction of higher-level experimental sequences differ substantially: PyProBE allows users to manually specify step information, while Cellpy and DATTES make use of schedule data provided in the imported cycler files to recreate step tables. However, a generalised approach which covers data without any step

DRAFT

information – as present in certain cyclers and especially field data – is missing from these packages.

PyDPEET integrates these functions in a common processing workflow. Data from different measurement systems is converted into a unified representation, elementary operating steps are identified from the measurements, and then combined into higher-level experimental sequences. Individual tests can subsequently be grouped into test series and measurements from multiple cells into campaigns while preserving the reconstructed structure. Thus, PyDPEET enables the analysis of heterogeneous datasets even when the original test schedule is unavailable.

Software Design

PyDPEET's main goal is to provide an easy-to-use, fast, and consistent library that can be easily integrated into existing workflows. Since many scientists in the field of battery research rely on Python scripts to read, analyse, and visualise their data, Python was chosen as the software's primary language. PyDPEET is already available as a package on PyPI and GitHub and provides an autogenerated API layer to give users top-level access to all relevant functions. The automatically updated GitHub Pages provide installation and development guidelines, an API reference generated from extensive docstrings, and in-depth tutorials covering the most common usecases.

To keep PyDPEET as lightweight as possible, the list of dependencies is kept at a minimum and additional resource files only consist of `.parquet` (Vohra, 2016) and `.txt` files for unit tests and "Sequence Analyzer" precompilation. The codebase itself uses a modular approach to allow future extension within existing submodules (see Fig. 1) as well as the addition of new submodules. The aforementioned top-level function calls ensure that users are unaffected by most internal changes.

Fig. 1 shows PyDPEET's general workflow. It uses a straightforward pipeline: first, input battery data is read, converted, and unified (`io` submodule); secondly, time series can be automatically divided into useful chunks for further analysis (`process/sequence` submodule); thirdly, data can be analysed using various functions for typical battery-related quantities (`process/analyze` submodule); lastly, evaluated data can be exported (`io` submodule). The "process/merge" submodule is an optional path for users who want to merge time series from multiple data files into a single file while retaining chronological order. All internal functionality is achieved using `Pandas dataframes` (McKinney, 2010; The pandas development team, 2020), while the standard output format is `.parquet`. The latter is a table-based format that is fast to process, enables high compression rates for typical input files in the MB-to-GB range, and can be easily converted into other typical formats, e.g., `.csv` or `.xlsx`.

DRAFT

The Journal of Open Source Software

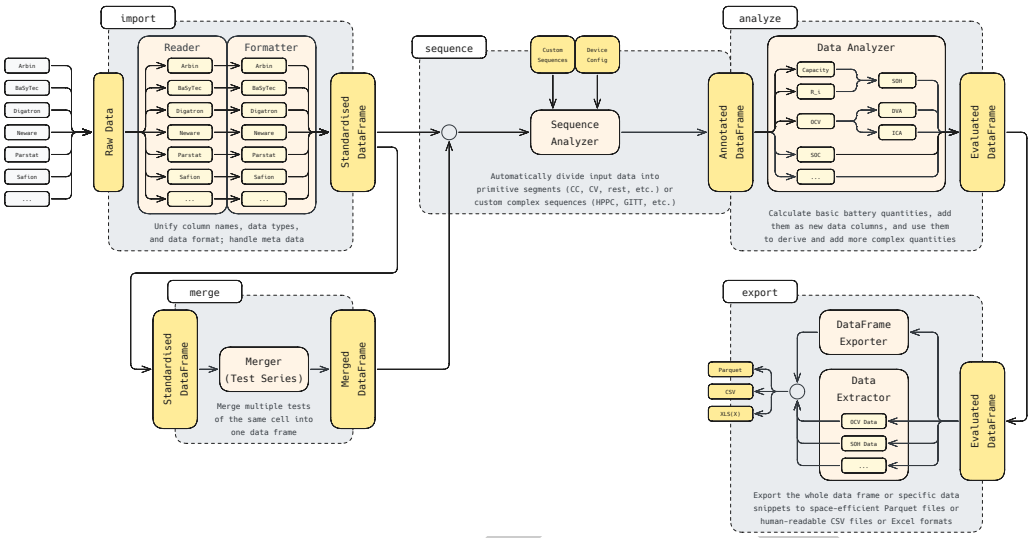


Figure 1: Overview of the structure and functionalities of the PyDPEET package

In addition to its functionality, a strong focus of the project is maintainability: Each commit triggers unit tests as well as a linting and formatting stage which uses Ruff and mypy to enforce formal code quality. The merge pipeline adds stages to build and deploy GitHub Pages for the latest state. Releases produce release-specific GitHub Pages and accompanying PyPI version updates.

The project integrates `citeme` (Berend Weel, 2017) to allow users to automatically aggregate scientific citations needed for their PyDPEET-based Python scripts. Required citations can be added at the function level to ensure that references only cover the functionality actually used in a script.

Research Impact Statement

PyDPEET has already been successfully adopted at the department of Electrical Energy Storage Technology (EET) at TU Berlin and is currently being used to process all existing experimental datasets of the department. It has also been used in several student theses, where it facilitated data analysis, produced reusable results, and was extended to fit the scope of certain thesis topics (see Acknowledgments).

The project was also presented as a poster at the Advanced Battery Power 2026 conference (Otto et al., 2026) and used to process and analyse the data used in another poster for the same conference (Schlösser et al., 2026), which were both met with great interest. The former was also shown at the offical opening of the Berlin Battery Lab to discuss cross-institutional usage within the alliance. Additionally, there are discussions with other research groups and within on-going projects to use PyDPEET for data handling and analysis. New algorithms resulting from these collaborations will be integrated into the PyDPEET ecosystem to further improve usage for all users.

AI Usage Disclosure

Generative AI tools were used to assist with code suggestions, debugging, documentation, website development, and improving the wording of the manuscript. All AI-assisted output was reviewed, validated, and revised by the authors.

DRAFT

Acknowledgements

This work was supported by the PODEST project, funded by the German Federal Ministry for Economic Affairs and Energy (BMWE) under grant number 03EI5280A, and the HELENA project, funded by the German Federal Ministry of Education and Research (BMBF) under grant number 03XP0534B.

The authors thank Dominik Droese for his continuous feedback and for providing measurement data from different test systems for the development and validation of PyDPEET. We also thank Felix Weidlich, Eleni Kiometzis, and Maximilian Thorn, who used PyDPEET in their bachelor theses and contributed to the project through testing, bug reports and fixes, and the implementation of additional functionality.

Battery Data Alliance. (2025). Battery-data-format. In *GitHub repository*. GitHub. https://github.com/battery-data-alliance/battery-data-format

Berend Weel, J. H. S. (2017). Citeme. In *GitHub repository*. GitHub. https://github.com/citation-file-format/citeme

Dubarry, M., Truchot, C., & Liaw, B. Y. (2012). Synthesize battery degradation modes via a diagnostic and prognostic model. *Journal of Power Sources*, *219*, 204–216. https://doi.org/10.1016/j.jpowsour.2012.07.016

Herring, P., Balaji Gopal, C., Aykol, M., Montoya, J. H., Anapolsky, A., Attia, P. M., Gent, W., Hummelshøj, J. S., Hung, L., Kwon, H.-K., Moore, P., Schweigert, D., Severson, K. A., Suram, S., Yang, Z., Braatz, R. D., & Storey, B. D. (2020). BEEP: A Python library for Battery Evaluation and Early Prediction. *SoftwareX*, *11*, 100506. https://doi.org/10.1016/j.softx.2020.100506

Holland, T., Cummins, D., & Marinescu, M. (2025). PyProBE: Python Processing for Battery Experiments. *Journal of Open Source Software*, *10*(106), 7474. https://doi.org/10.21105/joss.07474

Huang, J. D., & Zeier, W. G. (2026). Joint-Domain Impedance Spectroscopy for Solid-State Batteries: Enabling Accelerated Characterization and Data-Driven Insights. *ACS Energy Letters*, *11*(2), 986–994. https://doi.org/10.1021/acsenergylett.5c03055

McKinney, Wes. (2010). Data Structures for Statistical Computing in Python. In Stéfan van der Walt & Jarrod Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Murbach, M., Gerwe, B., Dawson-Elli, N., & Tsui, L. (2020). Impedance.py: A Python package for electrochemical impedance analysis. *Journal of Open Source Software*, *5*(52), 2349. https://doi.org/10.21105/joss.02349

Otto, M., Schlösser, A., Schröder, D., Simone, C. D., Hinrichsen, A., Kalisch, J., & Kowal, J. (2026, April). *PyDPEET: A Python Package for Fast and Easy Battery Data Unification, Processing, and Analysis*. https://doi.org/https://doi.org/10.13140/RG.2.2.35127.89768

Randall, C. (2025). *Ampworks: Battery analysis tools in Python [SWR-25-39]*. National Renewable Energy Laboratory (NREL), Golden, CO (United States). https://doi.org/10.11578/DC.20250313.2

Redondo-Iglesias, E., Hassini, M., Venet, P., & Pelissier, S. (2023). DATTES: Data analysis tools for tests on energy storage. *SoftwareX*, *24*, 101584. https://doi.org/10.1016/j.softx.2023.101584

Rehm, M., Natterer, J., Eizenhammer, J., Guenthner, M., Kuecher, S., Korkmaz, C., Roehrer, F., & Jossen, A. (2026). How to determine the degradation modes of lithium-ion batteries with silicon–graphite blend electrodes. *Journal of Power Sources*, *670*, 239418. https://doi.org/10.1016/j.jpowsour.2026.239418

DRAFT

Schlösser, A., Otto, M., & Kowal, J. (2026, April). *Automated OCV Extraction and Reconstruction from Highly Dynamic Field Data*. https://doi.org/https://doi.org/10.13140/RG.2.2.35966.75843

The pandas development team. (2020). *Pandas-dev/pandas: pandas* (latest). Zenodo. https://doi.org/10.5281/zenodo.3509134

Vohra, D. (2016). *Apache parquet* (pp. 325–335). https://doi.org/10.1007/978-1-4842-2199-0_8

Wan, T. H., Saccoccio, M., Chen, C., & Ciucci, F. (2015). Influence of the Discretization Methods on the Distribution of Relaxation Times Deconvolution: Implementing Radial Basis Functions with DRTtools. *Electrochimica Acta*, *184*, 483–499. https://doi.org/10.1016/j.electacta.2015.09.097

Ward, L. (2020). Battery-data-toolkit. In *GitHub repository*. GitHub. https://github.com/ROVI-org/battery-data-toolkit

Wind, J., Ulvestad, A., Abdelhamid, M., & Mæhlen, J. P. (2024). Cellpy – an open-source library for processing andanalysis of battery testing data. *Journal of Open Source Software*, *9*(97), 6236. https://doi.org/10.21105/joss.06236

DRAFT